Midterm DaAn

Ryan Bean

```r
remove(list = ls())

library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.1     v stringr   1.5.1
v ggplot2   4.0.0     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.1.0
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

```r
library(tidycensus)
```

```
Warning: package 'tidycensus' was built under R version 4.5.2
```

```r
library(fredr)
library(lubridate)
library(readxl)
library(readr)
library(janitor)
```

```


Attaching package: 'janitor'

The following objects are masked from 'package:stats':

    chisq.test, fisher.test
```

```r
library(tsibble)
```

```
Registered S3 method overwritten by 'tsibble':
  method               from
  as_tibble.grouped_df dplyr

Attaching package: 'tsibble'

The following object is masked from 'package:lubridate':

    interval

The following objects are masked from 'package:base':

    intersect, setdiff, union
```

```r
library(fpp3)
```

```
-- Attaching packages -------------------------------------------- fpp3 1.0.2 --
v tsibbledata 0.4.1     v fable       0.4.1
v feasts      0.4.2
-- Conflicts ------------------------------------------------- fpp3_conflicts --
x lubridate::date()    masks base::date()
x dplyr::filter()      masks stats::filter()
x tsibble::intersect() masks base::intersect()
x tsibble::interval()  masks lubridate::interval()
x dplyr::lag()         masks stats::lag()
x tsibble::setdiff()   masks base::setdiff()
x tsibble::union()     masks base::union()
```

```r
fredr_set_key("523a2b98a1ce120186357fd0c916cc26")
```

```r
years <- c(2005:2019, 2021:2024)

homeownership <- map_dfr(
  years,
  \(yr) {
    get_acs(
      geography = "state",
      variables = c(
```

```r
        total_occupied = "B25003_001",
        owner_occupied = "B25003_002"
      ),
      year = yr,
      survey = "acs1"
    ) %>%
      select(NAME, variable, estimate) %>%
      pivot_wider(names_from = variable, values_from = estimate) %>%
      mutate(
        year = yr,
        homeownership_rate = 100 * owner_occupied / total_occupied
      ) %>%
      transmute(
        state = NAME,
        year,
        homeownership_rate
      )
  }
)
```

```
Getting data from the 2005 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2006 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2007 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2008 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2009 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.
```

```
Getting data from the 2010 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2011 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2012 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2013 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2014 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2015 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2016 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2017 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2018 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2019 1-year ACS
```

```
The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2021 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2022 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2023 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2024 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.
```

```r
income_list <- map(
  years,
  ~ get_acs(
      geography = "state",
      variables = "B19013_001",
      year = .x,
      survey = "acs1"
    ) %>%
      transmute(
        state = NAME,
        year = .x,
        median_income = estimate
      )
)
```

```
Getting data from the 2005 1-year ACS
The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2006 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.
```

```
Getting data from the 2007 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2008 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2009 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2010 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2011 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2012 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2013 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2014 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2015 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2016 1-year ACS
```

```
The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2017 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2018 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2019 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2021 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2022 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2023 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2024 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.
```

```r
income <- bind_rows(income_list)

population_list <- map(
  years,
  ~ get_acs(
      geography = "state",
      variables = "B01003_001",
      year = .x,
      survey = "acs1"
    ) %>%
      transmute(
        state = NAME,
        year = .x,
        population = estimate
      )
)
```

```
Getting data from the 2005 1-year ACS
The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2006 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2007 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2008 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2009 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2010 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.
```

```
Getting data from the 2011 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2012 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2013 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2014 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2015 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2016 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2017 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2018 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2019 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2021 1-year ACS
```

```
The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2022 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2023 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.

Getting data from the 2024 1-year ACS

The 1-year ACS provides data for geographies with populations of 65,000 and greater.
```

```r
population <- bind_rows(population_list)

mortgage_raw <- fredr(
  series_id = "MORTGAGE30US",
  observation_start = as.Date("2005-01-01")
)

mortgage_annual <- mortgage_raw %>%
  mutate(year = year(date)) %>%
  group_by(year) %>%
  summarise(
    mortgage_rate = mean(value, na.rm = TRUE),
    .groups = "drop"
  )
```

```r
fhfa_raw <- read_csv("/Users/user/BostonCollege/DaAn Midterm/hpi_at_state (1).csv", col_names
  setNames(c("state", "year", "quarter", "hpi"))
```

```
Rows: 10404 Columns: 4
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (1): X1
dbl (3): X2, X3, X4

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
names(fhfa_raw)
```

```
[1] "state"   "year"    "quarter" "hpi"
```

```r
fhfa_annual <- fhfa_raw %>%
  clean_names() %>%
  filter(year >= 2005) %>%
  group_by(state, year) %>%
  summarise(
    hpi = mean(hpi, na.rm = TRUE),
    .groups = "drop"
  )

fhfa_annual <- fhfa_annual %>%
  mutate(state = state.name[match(state, state.abb)])
```

```r
bls_data <- read_delim("/Users/user/BostonCollege/DaAn Midterm/la.data.2.AllStatesU.txt", del
  clean_names()
```

```
Rows: 234289 Columns: 5
-- Column specification --------------------------------------------------------
Delimiter: "\t"
chr (4): series_id                      , period,        value, footnote_codes
dbl (1): year

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
bls_series <- read_delim("/Users/user/BostonCollege/DaAn Midterm/la.series.txt", delim = "\t
  clean_names()
```

```
Warning: One or more parsing issues, call `problems()` on your data frame for details,
e.g.:
  dat <- vroom(...)
  problems(dat)

Rows: 33881 Columns: 12
-- Column specification --------------------------------------------------------
Delimiter: "\t"
```

```
chr (9): series_id                     , area_type_code, area_code, measure_...
dbl (2): begin_year, end_year
lgl (1): footnote_codes

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
bls_area <- read_delim("/Users/user/BostonCollege/DaAn Midterm/la.area", delim = "\t") %>%
  clean_names()
```

```
Rows: 8325 Columns: 6
-- Column specification --------------------------------------------------------
Delimiter: "\t"
chr (3): area_type_code, area_code, area_text
dbl (2): display_level, sort_sequence
lgl (1): selectable

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# Join the actual data to the series metadata
bls_merged <- bls_data %>%
  left_join(bls_series, by = "series_id")

# Usually the common key is area_code
bls_merged <- bls_merged %>%
  left_join(
    bls_area %>% select(area_code, area_text),
    by = "area_code"
  )

# Keep annual-average unemployment rate
state_unemployment <- bls_merged %>%
  filter(period == "M13") %>%          # annual average
  filter(measure_code == "03") %>%     # unemployment rate
  transmute(
    state = area_text,
    year = as.integer(year),
    unemployment_rate = as.numeric(value)
  ) %>%
  filter(state %in% c(state.name, "District of Columbia")) %>%
  arrange(state, year)
```

```
Warning: There was 1 warning in `transmute()`.
i In argument: `unemployment_rate = as.numeric(value)`.
Caused by warning:
! NAs introduced by coercion
```

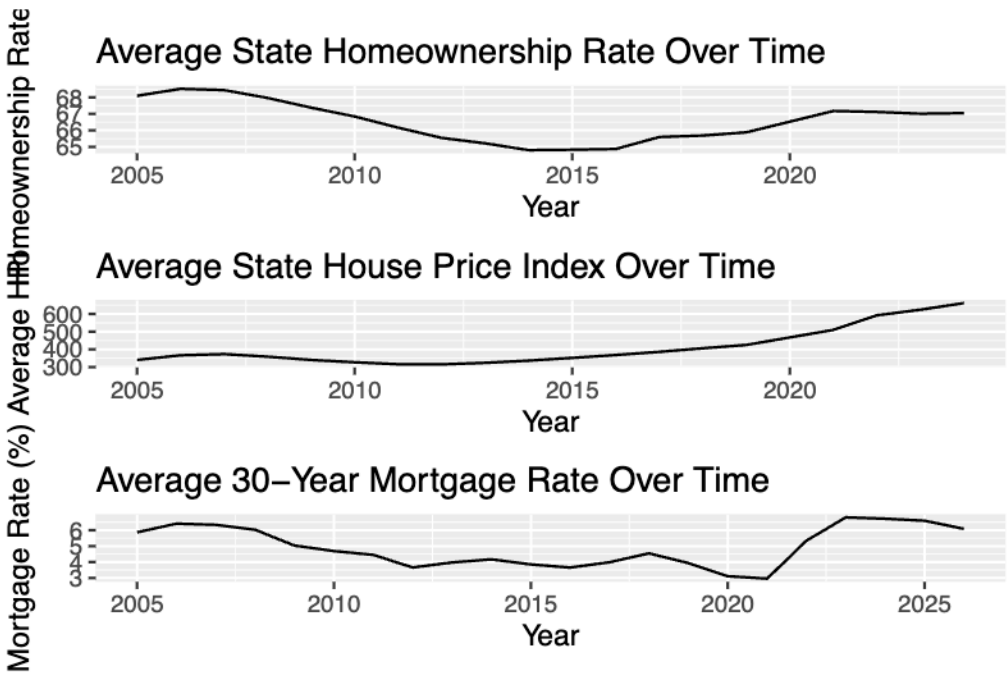
```r
state_unemployment
```

```
# A tibble: 2,499 x 3
   state    year unemployment_rate
   <chr>   <int>             <dbl>
 1 Alabama  1976               6.7
 2 Alabama  1977               7.1
 3 Alabama  1978               6.5
 4 Alabama  1979               7.2
 5 Alabama  1980               8.7
 6 Alabama  1981              10.8
 7 Alabama  1982              13.8
 8 Alabama  1983              13.8
 9 Alabama  1984              11.4
10 Alabama  1985               9.6
# i 2,489 more rows
```

```r
panel_data <- homeownership %>%
  left_join(income, by = c("state", "year")) %>%
  left_join(population, by = c("state", "year")) %>%
  left_join(fhfa_annual, by = c("state", "year")) %>%
  left_join(state_unemployment, by = c("state", "year")) %>%
  left_join(mortgage_annual, by = "year") %>%
  arrange(state, year)

panel_data %>%
  summarise(across(everything(), ~ sum(is.na(.))))
```

```
# A tibble: 1 x 8
  state  year homeownership_rate median_income population   hpi
  <int> <int>              <int>         <int>      <int> <int>
1     0     0                  0             0          0    38
# i 2 more variables: unemployment_rate <int>, mortgage_rate <int>
```

```r
panel_ts <- panel_data %>%
  as_tsibble(key = state, index = year)
```

```r
library(patchwork)

# 1. Average homeownership rate over time
avg_homeownership_ts <- panel_data %>%
  group_by(year) %>%
  summarise(
    avg_homeownership_rate = mean(homeownership_rate, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  as_tsibble(index = year)

p1 <- autoplot(avg_homeownership_ts, avg_homeownership_rate) +
  labs(
    title = "Average State Homeownership Rate Over Time",
    x = "Year",
    y = "Homeownership Rate (%)"
  )

# 2. Average HPI over time
avg_hpi_ts <- panel_data %>%
  group_by(year) %>%
  summarise(
    avg_hpi = mean(hpi, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  as_tsibble(index = year)

p2 <- autoplot(avg_hpi_ts, avg_hpi) +
  labs(
    title = "Average State House Price Index Over Time",
    x = "Year",
    y = "Average HPI"
  )

# 3. Mortgage rate over time
mortgage_ts <- mortgage_annual %>%
  as_tsibble(index = year)

p3 <- autoplot(mortgage_ts, mortgage_rate) +
```

```r
  labs(
    title = "Average 30-Year Mortgage Rate Over Time",
    x = "Year",
    y = "Mortgage Rate (%)"
  )

# Combine all three plots
p1 / p2 / p3
```